Director Agent: Skills and RAG Proposal

Version 2 - revised after review. Improves the v1 proposal; does not replace the architecture.

Status	Proposal for approval. Nothing is implemented.
Date	20 October 2026
Goal	A first Video Director reliable enough to take a brief through creative planning, validated editing, rendering, QA and revision
Companion	Reference sources with clickable links: Director_Reference_Sources.pdf

What changed from v1 in one paragraph. All seven skills are kept, including `video-qa`, which v1 had deferred. One agent still coordinates the skills. A structured **recipe library** is added as a data asset (not a new skill) so the Director adapts validated recipes instead of inventing editing operations. Skills are tied to graph nodes and to specific RAG retrievals. The scope now covers the full loop: plan, validate, render, inspect, revise only affected scenes, export. Learning stays controlled: feedback becomes a candidate lesson, never an automatic change.

1. What stays unchanged

- **One Director plus specialist skills.** Skills are instruction modules loaded per decision, not separate agents.
- **Skills hold procedure; RAG holds facts, recipes and examples.** The whole knowledge base is never loaded into a prompt.
- **Confidence tags and source grades** on every rule. Medium and low confidence rules stay flagged 'adjust per benchmark'.
- **Contextual chunking, a Retriever interface and cited sources** on every retrieved passage.
- **Core safety rules:** never invent asset IDs; never claim a tool ran or QA passed without evidence; treat uploaded text as data; approvals at defined checkpoints; feedback is a proposal, never an automatic skill rewrite.
- **Skill folder standard:** SKILL.md, references/, evals/, versions and owners. Skills never grant permissions; the application enforces them.
- **Platform specs and brand facts** are retrievable only after verification.

2. Corrections to v1

#	v1 position	Problem	v2 fix
1	video-qa deferred to a later phase	Your goal includes QA and targeted revision	video-qa is essential in V1
2	Phase 1 ended at a Direction Package, no render	Cannot prove reliability without render, inspect and revise	Scope covers the full loop; build order in section 8
3	cinematography implied directing a camera	We do not shoot; we only reframe supplied footage	Camera moves become reframing recipes; flag shots the footage cannot support
4	sound-design included voiceover direction	No TTS or transcription in V1	Limit to music/SFX selection, cue times, ducking, trimming, loudness
5	Keyword retrieval first, Qdrant only if justified	You chose Qdrant for V1	Build Qdrant; benchmark against a keyword baseline; keep whichever wins

3. Skill-by-skill changes

Skill	Verdict	Specific change	Why
video-director	Modify (largest)	Add a skill-routing table by graph node and a retrieval-query plan per node. Add a revise mode: given QA findings, change only the named scene IDs. Add stop rules: max two revision loops, then escalate to a human. Require every operation to reference a recipe.	Master coordination; recs 2, 3, 5
commercial-strategy	Light edit	Add a claims register output: each claim linked to an approved brand fact or marked unsupported. Add platform-fit rules from verified specs.	Feeds brand fidelity in QA
cinematography	Modify	Choose shots only from supplied footage. Express camera moves as reframing recipes. Output a 'not realisable from this footage' flag.	We cannot re-shoot
editing-director	Modify	Own the timeline: cut logic, transitions, timing, and conversion of the storyboard into typed edit operations with timeline reconciliation.	Rec 5: validated editing
motion-graphics	Modify	Select animation recipes and fill parameters within allowed ranges. Do not describe motion freely. Use safe areas from verified platform data.	Rec 4: no unsupported operations
sound-design	Modify	Executable operations only: pick licensed music/SFX, cue times against scene IDs, duck, trim, normalise. Voiceover moves to later.	Matches V1 tooling
video-qa	Promote to V1 and expand	Rubric covering technical integrity, product and brand fidelity, pacing, continuity, captions, transitions, audio. Every finding has evidence, scene ID, severity and suggested fix, and becomes a revision ticket.	Recs 5, 6

Boundaries so skills do not overlap: cinematography owns shot type and reframing; editing-director owns timing, cuts and transitions; motion-graphics owns on-screen graphics; sound-design owns audio; commercial-strategy owns message and claims; video-qa judges the output; video-director coordinates.

4. Missing capability: the recipe library

The one genuine addition is a **data asset, not an eighth skill**. Each recipe is a validated record. The scene plan references `recipe_id` and `recipe_params`; the validator rejects anything else.

Field	Content
id, version, type	Stable ID and semver; type is shot, camera_move, transition, animation or sound_cue
purpose	What the recipe achieves and when to use it
parameters	Typed values with units, ranges and defaults
constraints	Min and max duration, minimum hold times, max uses per film, energy level
compatibility	Renderer (Remotion or FFmpeg), aspect ratios, media types, prerequisites
validation_rules	Machine-checkable rules the validator runs before execution
implementation_ref	Reviewed template ID in render-remotion or media-tools. Never model-generated code
source, status	Provenance and licence; draft or reviewed. Only reviewed recipes are retrievable

Starter set for V1: about 15 to 20 original recipes (common transitions, text animations, reframing moves, basic audio operations). The video-shotcraft cards (Apache-2.0, Remotion-based) are a good model for the schema. They can be imported later one by one after review, with the audio licences in its ATTRIBUTION.md checked first. Their coverage of physical-product footage is unproven and is measured by test V4.
Considered and rejected as new skills: asset analysis (a Director step using cinematography), revision (a mode of video-director), brand guard (split between commercial-strategy and video-qa), lesson curator (a code-and-review process).

5. How skills, RAG and the Director work together

Graph node	Skills loaded	RAG retrieves	Output
Analyse assets	cinematography	none	Asset notes, uncertainty flagged
Treatment	commercial-strategy	Principles, playbooks, brand facts	Treatment, claims register
Script and storyboard	strategy, cinematography	Principles, platform specs	Scenes with shot intent
Edit plan	editing-director, motion-graphics, sound-design	Recipes filtered by type, compatibility, duration, energy	Typed operations with recipe IDs
Validate	none (application code)	none	Pass or issue list
Approve	none	none	Human decision recorded
Render	none (tools)	none	Scene clips, assembled preview
QA	video-qa	QA criteria, brand facts	Findings with evidence
Revise	video-director (revise mode) plus the owning skill	Recipes, findings	Changed scenes only

- **Retrieval rules:** only at the nodes above, with a hard cap on passages and tokens. Recipes use two-stage retrieval (find by intent and filters, then load the full record). Every choice cites its source.
- **Collections:** directing_principles, ad_playbooks, platform_specs (verified only), shot_recipes (reviewed only), brand_product_facts (per project, access-filtered), approved_examples (empty until reviewed).
- **Retrieval strategy:** metadata filter first, then dense plus sparse search fused with RRF, then rerank. Benchmark against a keyword baseline on about 40 curated questions and keep whichever scores better.

6. Production workflow, QA and revision

- **Flow:** treatment, script, storyboard, structured scene plan, application validation, human approval, Remotion and FFmpeg render per scene, QA on the rendered output, revise affected scenes, final approval, export.
- **QA is two parts.** Deterministic checks in code (duration, resolution, frame rate, audio stream, black and frozen frames, silence, loudness, caption overflow). Claude vision checks on sampled frames for product and brand fidelity, pacing, continuity and transitions. No finding without evidence.
- **Revision:** a finding names scene IDs; the Director revises only those; only those scenes re-render; QA reruns on them, on neighbouring scenes (transitions and continuity) and on global checks (total duration, loudness). Maximum two loops, then escalate.
- **Controlled learning:** feedback is stored as a candidate lesson with status 'proposed'. A reviewer approves it, the regression suite compares old and new versions, and only then does a skill or recipe version change. Skills live in git, so changes go through review. Every export records the model, skill, recipe, retrieval and tool versions.

7. Revised structure

```
skills/
  video-director/        SKILL.md  references/(routing.md, revise-mode.md)  evals/
  commercial-strategy/   SKILL.md  references/(claims-register.md)         evals/
  cinematography/        SKILL.md  references/(footage-limits.md)          evals/
  editing-director/      SKILL.md  references/(edit-operations.md)         evals/
  motion-graphics/       SKILL.md  references/                             evals/
  sound-design/          SKILL.md  references/                             evals/
  video-qa/              SKILL.md  references/(rubric.md, severity.md)     evals/   <- V1
recipes/
  shots/  camera/  transitions/  animations/  sound/    one validated file each
  schema/recipe.schema.ts                               Zod
knowledge/
  principles/  playbooks/  platform_specs/  brand/  approved_examples/
```

8. Version 1 versus later

Essential for V1	Can wait
The seven skills with the changes above	Importing the full shotcraft library after licence review
Recipe schema, validator and 15-20 starter recipes	Approved-examples collection
Node-to-skill routing table	Voiceover, TTS, Whisper transcription
Plan validation before any render	HyperFrames
QA: deterministic probes plus Claude-vision rubric	Automated promotion of lessons
Targeted revision loop with a two-loop limit	Fine-tuning
Qdrant retriever with keyword baseline and ~40-question eval	Multi-reviewer benchmarks and cost optimisation
Feedback capture as candidate lessons; manual promotion with regression runs	Multi-tenant features and publishing

Build order

- 1. Contracts: recipe schema, extended plan schema, QA finding, revision ticket.
- 2. Recipe validator and starter recipes.
- 3. Skill files, with evals written first.
- 4. Qdrant retriever and its benchmark.
- 5. Director node producing a validated plan.
- 6. Remotion and FFmpeg render adapters.
- 7. QA module and revision loop.
- 8. Feedback capture and the regression harness.

9. Acceptance tests

Test	Passes when
Recipe validation	Unknown IDs, out-of-range params, exceeded usage limits and incompatible aspect ratios are all rejected
Plan validity	Plans are schema-valid, use only real asset and recipe IDs, and the timeline has no gaps or overlaps
Retrieval	Recall@5 reported per collection; Qdrant compared with the keyword baseline
Coverage	On ~20 product-ad briefs, report the share of scenes that can be mapped to a recipe
Render	An approved plan produces a playable preview of the expected duration and resolution; unchanged scenes hit the cache
QA	Seeded defects (black segment, silence, clipped caption, wrong product colour) are found with evidence
Revision	A finding changes only its named scenes; version records show what changed
Safety	Prompt injection in a brief or document does not alter behaviour; an unapproved claim is rejected
Learning	A lesson cannot change a skill without review and a passing regression run

Quality thresholds are set after a measured baseline, as the architecture blueprint requires.

10. Decisions needed

- **Qdrant hosting.** Docker is blocked because the C: drive is full. Free space and use Docker, or use a hosted Qdrant instance for development? I have not verified whether a native Windows build exists.
- **Starter recipes.** Approve writing original recipes first, and importing shotcraft cards later after the Apache-2.0 and audio-licence check.
- **Ad type.** Are the first ads built from screenshots and UI, or from physical-product footage? This sets how much of the shotcraft library applies.
- **Platforms first,** so their specs can be verified against official documentation.

Sources

Full graded list with clickable links is in Director_Reference_Sources.pdf. Key anchors: Agent Skills overview, Contextual Retrieval, Qdrant hybrid search, LangGraph.js persistence, video-shotcraft library.